

Chapter 7: Input and Output

The standard library's I/O model: streams, formatted conversion, file access, and error handling.

How to use this document

Read the graph first. Each node is a concept; each edge is a dependency. Walk the edges in order. After each edge, do the green problem immediately. Checkpoints (red) tell you whether you can proceed. The target problem at the end ties every edge together. Answer key is on the last page.

Big picture

C itself has no I/O keywords. Everything goes through the standard library declared in `<stdio.h>`. The central abstraction is `FILE*` -- an opaque pointer to a stream. Three streams exist at startup: `stdin`, `stdout`, `stderr`. You open more with `fopen`, read/write through them, and close them with `fclose`.

Target problem

Write a program that reads from a named file (or `stdin` if no file argument is given), converts tabs to spaces (using a configurable tab stop width), and writes to `stdout` with error reporting to `stderr`.

By the end of this document you will write this from scratch, understanding every line.

The Graph

Node inventory

Node	What it is	Why it matters
<code>FILE*</code>	Opaque pointer to a stream object	Every I/O operation needs one
<code>stdin</code> / <code>stdout</code> / <code>stderr</code>	Three pre-opened <code>FILE*</code> streams	Available without <code>fopen</code> ; <code>stderr</code> is unbuffered
<code>fopen</code> / <code>fclose</code>	Open a named file / release the stream	The lifecycle brackets for file I/O
<code>printf</code> / <code>fprintf</code> / <code>sprintf</code>	Formatted output to <code>stdout</code> / file / string	Convert internal values to text via format strings

scanf / fscanf / sscanf	Formatted input from stdin / file / string	Parse text into typed variables via format strings
getc / putc	Read / write one character at a time	Lowest-level stream I/O; may be macros
fgets / fputs	Read / write one line at a time	Safer than gets; respects buffer size
exit()	Terminate program with status code	Flushes all streams, returns status to OS

Edge table

From	To	Relationship
FILE*	fopen / fclose	fopen creates a FILE*, fclose destroys it -- lifecycle
FILE*	getc / putc / fgets / fputs	All character/line I/O requires a FILE*
FILE*	fprintf / fscanf	Formatted I/O targeting a specific stream
format strings	printf family	The format string controls conversion of every argument
format strings	scanf family	Same format concept, opposite direction (parsing)
stderr	exit()	Error path: report to stderr, then exit with nonzero status
stdin/stdout	getc/putc with stdin/stdout	getchar() = getc(stdin); putchar(c) = putc(c, stdout)

Zoom in -- one edge at a time

Edge 1: stdin/stdout with character-at-a-time I/O

The simplest I/O model: read one character with `getchar()`, write one with `putchar()`. Both operate on the default streams. `getchar` returns `int` (not `char`) so it can represent EOF, which is typically -1.

```
#include <stdio.h>
#include <ctype.h>

/* Convert input to uppercase, char by char */
int main(void) {
    int c;
    while ((c = getchar()) != EOF)
        putchar(toupper(c));
    return 0;
}
```

Do now: Write a program that counts the number of lines, words, and characters from stdin (a minimal wc). Print the three counts to stdout.

Edge 2: format strings control printf output

A format string mixes literal text with conversion specifiers: `%d` (int), `%f` (double), `%s` (string), `%c` (char), `%x` (hex), `%o` (octal). Width and precision: `%10.2f` means field width 10, 2 decimal places. Minus sign means left-justify: `%-20s`.

```
int n = 255;
double pi = 3.14159265;
char *label = "result";

printf("%-10s %d 0x%04x %.4f\n", label, n, n, pi);
/* Output: result 255 0x00ff 3.1416 */

/* sprintf writes to a string buffer instead of stdout */
char buf[100];
sprintf(buf, "Item %d costs $%.2f", 3, 9.99);
```

Do now: Write a function that prints an array of doubles as a right-aligned table with an index column. Format: `[ 0] 3.1416`. Use field widths.

Edge 3: scanf parses input using format strings

`scanf` reads from stdin; `fscanf` reads from a `FILE*`; `sscanf` reads from a string. Each returns the number of items successfully matched. Arguments must be pointers. The classic mistake: writing `scanf("%d", n)` instead of `scanf("%d", &n)`.

```
int day, year;
char month[20];
```

```

/* Parse "25 Dec 1988" */
if (scanf("%d %s %d", &day, month, &year) == 3)
    printf("Parsed: %s %d, %d\n", month, day, year);

/* sscanf: parse from a string */
char *line = "3.14 2.71";
double a, b;
sscanf(line, "%lf %lf", &a, &b);

```

Do now: Read lines of the form NAME AGE GPA from stdin using fgets + sscanf. Print each parsed record. Stop at EOF. Handle the case where sscanf does not match all 3 fields.

Edge 4: FILE* lifecycle -- fopen, use, fclose

fopen(name, mode) returns a FILE* or NULL on failure. Modes: "r" read, "w" write (truncates), "a" append. After use, fclose flushes buffers and releases the stream. Always check fopen's return value.

```

FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    fprintf(stderr, "error: cannot open data.txt\n");
    exit(1);
}
int c;
while ((c = getc(fp)) != EOF)
    putc(c, stdout);
fclose(fp);

```

Do now: Write a program that takes a filename as argv[1], opens it, and prints it line by line using fgets. If no argument is given, read from stdin instead. Use the same fgets call for both paths.

Edge 5: stderr and exit for error handling

stderr is a third stream, separate from stdout. It is typically unbuffered, so error messages appear immediately even when stdout is redirected to a file. exit(n) terminates the program, flushing all open streams. Convention: exit(0) = success, nonzero = failure.

```

/* Pattern: open-or-die */
FILE *safe_open(char *name, char *mode, char *prog) {
    FILE *fp = fopen(name, mode);
    if (fp == NULL) {
        fprintf(stderr, "%s: cannot open %s\n", prog, name);
        exit(1);
    }
    return fp;
}

```

Do now: Modify your previous file-printer to: (a) include `argv[0]` in error messages, (b) use `fprintf(stderr, ...)` for all errors, (c) `exit(1)` on failure, `exit(0)` on success.

Edge 6: `fgets/fputs` for line-oriented I/O

`fgets(buf, size, fp)` reads up to `size-1` characters or until newline (which is kept). Returns `NULL` on EOF or error. `fputs(str, fp)` writes a string (no added newline). Prefer `fgets` over `gets`, which has no buffer-size limit and is unsafe.

```
/* Copy file line by line */
void filecopy(FILE *in, FILE *out) {
    char line[4096];
    while (fgets(line, sizeof(line), in) != NULL)
        fputs(line, out);
}
```

Do now: Write a program that numbers each line of input. Read from a file if given, otherwise `stdin`. Output format: `1: line text here.`

CHECKPOINT A

Before continuing, verify you can answer these without looking back:

1. Why does `getchar` return `int` instead of `char`?
2. What does `sprintf` write to?
3. What happens if you call `fprintf(stderr, ...)` and `stdout` is redirected to a file?
4. What is the danger of `gets()` compared to `fgets()`?

Solve the target -- step by step

Build a tab-to-spaces converter that reads from a named file (or stdin), writes to stdout, and reports errors to stderr.

Step 1: Parse arguments

```
#include <stdio.h>
#include <stdlib.h>

#define TABSTOP 8

int main(int argc, char *argv[]) {
    FILE *fp;
    char *prog = argv[0];

    if (argc == 1)
        fp = stdin; /* no args: read stdin */
    else if (argc == 2) {
        fp = fopen(argv[1], "r");
        if (fp == NULL) {
            fprintf(stderr, "%s: cannot open %s\n", prog, argv[1]);
            exit(1);
        }
    } else {
        fprintf(stderr, "usage: %s [filename]\n", prog);
        exit(1);
    }
}
```

Step 2: Tab expansion loop

```
int c;
int col = 0; /* current column position */

while ((c = getc(fp)) != EOF) {
    if (c == '\t') {
        int spaces = TABSTOP - (col % TABSTOP);
        int i;
        for (i = 0; i < spaces; i++)
            putchar(' ');
        col += spaces;
    } else if (c == '\n') {
        putchar(c);
        col = 0;
    } else {
        putchar(c);
        col++;
    }
}
```

Step 3: Cleanup and error check

```
if (fp != stdin)
    fclose(fp);
```

```
if (ferror(stdout)) {
    fprintf(stderr, "%s: error writing stdout\n", prog);
    exit(2);
}
return 0;
}
```

Key observations:

- `getc(fp)` works identically whether `fp` is `stdin` or a named file.
- Errors go to `stderr` so they appear on screen even if `stdout` is piped.
- `ferror(stdout)` catches write failures (e.g., disk full, broken pipe).
- Column tracking resets on newline. Tab expansion depends on current column.

CHECKPOINT B

1. Why does the tab expansion need to know the current column?
2. Why do we check `ferror(stdout)` at the end?
3. Why close `fp` only if it is not `stdin`?

Extensions

Extension 1: Configurable tab width

Accept an optional `-t N` flag before the filename. Parse it from `argv`. Default to 8 if not given.

Extension 2: Multiple files

Accept multiple filenames. Process each in order, like `cat`. If any file fails to open, print an error to `stderr` but continue with the rest. Track whether any error occurred and set the exit status accordingly.

Extension 3: Reverse operation (spaces to tabs)

Write the inverse: collapse runs of spaces at tab-stop boundaries back into tabs. This requires buffering spaces and deciding at each tab stop whether to emit a tab.

Extension 4: Variable-length argument processing

Write a function `void errmsg(char *fmt, ...)` using `<stdarg.h>` that prefixes all output with the program name and sends it to `stderr`. Use `va_start`, `va_arg` pattern from Chapter 7.3.

CHECKPOINT C

1. How would you parse `-t N` from `argv` before the filename arguments?
2. In multi-file mode, should you `exit(1)` on the first error or continue?
3. In the spaces-to-tabs reverse, why do you need to buffer spaces?

Final exam -- 5 problems

Problem 1. Write a program that concatenates multiple files to stdout (a simplified cat). If no files are given, copy stdin. Report errors to stderr with the program name. Return nonzero exit status if any file could not be opened.

Problem 2. Write a formatter that reads lines from stdin and prints them right-justified to a given width (e.g., 60 columns). Use `sprintf` to build the output string. Lines longer than the width are printed as-is.

Problem 3. Write a program that reads a CSV file (comma-separated) and prints it as a formatted table. Use `fgets` to read lines, `sscanf` or manual parsing to split fields, and `printf` with field widths to align columns.

Problem 4. Implement `void errprintf(char *fmt, ...)` using `<stdarg.h>` that behaves like `fprintf(stderr, fmt, ...)` but automatically prepends the program name. Write a test driver that calls it with different argument types (int, string, double).

Problem 5. Write a file-comparison program: given two filenames, read both line by line with `fgets`. Print the first line number and content where they differ. If one file is shorter, report that. Report all errors to stderr and exit with appropriate status codes.

CHECKPOINT D

If you completed all 5 problems, you have working fluency with: `FILE*` lifecycle (`fopen/fclose`), character I/O (`getc/putc`), line I/O (`fgets/fputs`), formatted I/O (`printf/scanf` families), error handling (`stderr/exit/error`), and variable-length argument lists. These cover all standard C I/O operations.

Answer key

Problem 1 (simplified cat)

```
#include <stdio.h>
#include <stdlib.h>

void filecopy(FILE *in, FILE *out) {
    int c;
    while ((c = getc(in)) != EOF)
        putc(c, out);
}

int main(int argc, char *argv[]) {
    char *prog = argv[0];
    int err = 0;

    if (argc == 1) {
        filecopy(stdin, stdout);
    } else {
        int i;
        for (i = 1; i < argc; i++) {
            FILE *fp = fopen(argv[i], "r");
            if (fp == NULL) {
                fprintf(stderr, "%s: cannot open %s\n",
                    prog, argv[i]);
                err = 1;
                continue;
            }
            filecopy(fp, stdout);
            fclose(fp);
        }
        if (ferror(stdout)) {
            fprintf(stderr, "%s: error writing stdout\n", prog);
            err = 2;
        }
        return err;
    }
}
```

Problem 2 (right-justify)

```
#include <stdio.h>
#include <string.h>

#define WIDTH 60

int main(void) {
    char line[1024];
    while (fgets(line, sizeof(line), stdin)) {
        /* Strip trailing newline */
        int len = strlen(line);
        if (len > 0 && line[len-1] == '\n')
            line[--len] = '\0';
```

```

if (len <= WIDTH)
printf("%s\n", WIDTH, line);
else
printf("%s\n", line);
}
return 0;
}

```

Problem 4 (errprintf with stdarg)

```

#include <stdio.h>
#include <stdarg.h>

static char *programe = "myapp";

void errprintf(char *fmt, ...) {
va_list ap;
fprintf(stderr, "%s: ", programe);
va_start(ap, fmt);
vfprintf(stderr, fmt, ap);
va_end(ap);
}

/* Test: */
/* errprintf("failed to open %s (code %d)\n", fname, errno); */

```

Problem 5 (file compare)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
if (argc != 3) {
fprintf(stderr, "usage: %s file1 file2\n", argv[0]);
exit(1);
}
FILE *f1 = fopen(argv[1], "r");
FILE *f2 = fopen(argv[2], "r");
if (!f1 || !f2) {
fprintf(stderr, "%s: cannot open input\n", argv[0]);
exit(1);
}
char l1[4096], l2[4096];
int lineno = 0;
while (fgets(l1, sizeof l1, f1) && fgets(l2, sizeof l2, f2)) {
lineno++;
if (strcmp(l1, l2) != 0) {
printf("differ at line %d:\n %s %s", lineno, l1, l2);
fclose(f1); fclose(f2);
return 1;
}
}
}

```

```
if (!feof(f1) || !feof(f2))
    printf("files differ in length after line %d\n", lineno);
else
    printf("files are identical\n");
fclose(f1); fclose(f2);
return 0;
}
```

Problems 3 (CSV table)

Read each line with `fgets`. Split on commas manually (walk the string with a pointer, copy characters to a field buffer until comma or newline). Store fields in a 2D array. First pass: find max width per column. Second pass: `printf` with `%- *s` to left-justify each field at the computed width.